Dfs.py

```python
def dfs(graph,node,visited):
    print('visited',node)
    visited.add(node)

    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs(graph,neighbour,visited)


graph={
    'A':['B','C'],
    'B':['D','E'],
    'C':['F'],
    'D':[],
    'E':[],
    'F':[]
}

visited=set()
print('DFS Traversal: ')
dfs(graph, node: 'A',visited)
```

Bfs.py

```python
def bfs(graph,start):
    visited=[start]
    queue=[start]
    print("BFS:")

    while queue:
        node=queue.pop(0)
        print(node,end=" ")
        for n in graph[node]:
            if n not in visited:
                visited.append(n)
                queue.append(n)

def shortest_path(graph,start,end):
    queue=[[start]]
    visited=[]

    while queue:
        path=queue.pop(0)
        node=path[-1]

        if node==end:
            return path

        if node not in visited:
            for n in graph[node]:
                queue.append(path+[n])
            visited.append(node)

graph={
'A':['B','C'],
'B':['A','D'],
'C':['A','D'],
'D':['B','C']
}

bfs(graph, start: 'A')
print("\nShortest Path:",shortest_path(graph, start: 'A', end: 'D'))
```

A star.py

```python
import heapq
graph = {
'A':[('B',1),('C',3)],
'B':[('D',1),('E',5)],
'C':[('F',2)],
'D':[('F',2)],
'E':[],
'F':[]
}

h = {'A':6,'B':4,'C':4,'D':0,'E':0,'F':0}

def a_star(start, goal):
    pq = [(h[start], 0, start, [])]   # (f, cost, node, path)
    visited = set()

    while pq:
        f, cost, node, path = heapq.heappop(pq)

        if node in visited:
            continue

        visited.add(node)
        path = path + [node]

        if node == goal:
            return path, cost

        for neighbour, weight in graph[node]:
            heapq.heappush(
                pq,
                _item: (cost + weight + h[neighbour], cost + weight, neighbour, path)
            )

path, cost = a_star( start: 'A', goal: 'F')

print("Path:", path)
print("Cost:", cost)
```

AO Star.py

```python
# AND-OR Graph
graph = {
    'A': [[('B',1),('C',1)], [('D',1)]],
    'B': [[('E',1)], [('F',1)]]
}

# Heuristic values
h = {'A':10,'B':6,'C':2,'D':4,'E':1,'F':3}


def ao(n):
    if n not in graph:
        return h[n]
    h[n]=min(sum(w+ao(c) for c,w in group)
             for group in graph[n]

             )
    return h[n]

print("Optimal cost:", ao('A'))

```

TowerOfHanoi.py

```python
def movedisk(ip,tp):
    print('disk is moved from: ',ip,'to',tp)

def movetower(n,ip,tp,withpole):
    if n==1:
        movedisk(ip,tp)
        return

    movetower(n-1,ip,withpole,tp)
    movedisk(ip,tp)
    movetower(n-1,withpole,tp,ip)

n=int(input("enter a number: "))
movetower(n, ip: 'A', tp: 'C', withpole: 'B')

```

Missionaries & Cannibals.py

```python
from collections import deque

start = (3,3,0)
goal = (0,0,1)

moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]

queue = deque([(start,[start])])
visited = set()

while queue:
    (m,c,b), path = queue.popleft()

    if (m,c,b) == goal:
        print("Solution:")
        for step in path:
            print(step)
        break

    if (m,c,b) in visited:
        continue

    visited.add((m,c,b))

    for dm,dc in moves:
        if b == 0:
            nm,nc,nb = m-dm, c-dc, 1
        else:
            nm,nc,nb = m+dm, c+dc, 0

        if 0<=nm<=3 and 0<=nc<=3:
            rm,rc = 3-nm, 3-nc

            if (nm==0 or nm>=nc) and (rm==0 or rm>=rc):
                queue.append(((nm,nc,nb), path+[(nm,nc,nb)]))
```

Tic Tac Toe.py

```python
import tkinter as tk
from tkinter import messagebox

root = tk.Tk()
root.title("Tic Tac Toe")

board=[""]*9
player="X"

def check():
    win=[(0,1,2),(3,4,5),(6,7,8),
         (0,3,6),(1,4,7),(2,5,8),
         (0,4,8),(2,4,6)]
    return any(board[a]==board[b]==board[c]!="" for a,b,c in win)

def click(i):
    global player
    if board[i]==="":
        board[i]=player
        btn[i].config(text=player)

        if check():
            messagebox.showinfo( title: "Winner",player+" wins")
            root.quit()
        elif "" not in board:
            messagebox.showinfo( title: "Draw", message: "Match Draw")
            root.quit()
        else:
            player="O" if player=="X" else "X"

btn=[]
for i in range(9):
    b=tk.Button(root,text="",font=("Arial",20),
                width=5,height=2,
                command=lambda x=i: click(x))
    b.grid(row=i//3,column=i%3)
    btn.append(b)
root.mainloop()
```